# VEHICLE RENTAL SYSTEM

## C++ Object-Oriented Programming Mini-Project Report

## 1. Introduction

The Vehicle Rental System is a menu-driven C++ console application. It allows a user to select a car or a bike, enter basic rental information, and view the calculated rental charges.

## 2. Objectives

• Apply classes and objects to model vehicles.
• Demonstrate single inheritance using a common Vehicle base class.
• Use function overriding and virtual functions for vehicle-specific rent calculation.
• Validate menu choices, vehicle IDs, and rental duration.
• Display a clear rental receipt.

## 3. Class Design

**Vehicle (base class):** stores vehicle ID, model, and daily rental rate. It provides virtual functions for calculating rent, identifying the vehicle type, and displaying details.
**Car (derived class):** inherits Vehicle and adds a fixed Rs. 200 insurance charge. Its rent is daily rate × days + insurance charge.
**Bike (derived class):** inherits Vehicle. Its rent is daily rate × days.

## 4. OOP Concepts Used

**Class and Object:** Classes define the structure and behaviour; objects represent the selected vehicle.
**Inheritance:** Car and Bike publicly inherit common data and behaviour from Vehicle.
**Encapsulation:** Vehicle data members are kept in the protected section and are accessed through class methods/derived-class behaviour.
**Constructors:** Constructors initialize ID, model, and rental rate.
**Function Overriding:** Car and Bike provide their own implementation of calculateRent().
**Runtime Polymorphism:** A base-class pointer refers to a Car or Bike object; virtual calls execute the matching derived-class function.
**Virtual Destructor:** Supports safe deletion of derived objects through a base pointer.

## 5. Rental Calculation

Car: (Rs. 1,500 × number of days) + Rs. 200 insurance.
Bike: Rs. 500 × number of days.
These are illustrative rates selected for the mini-project and are not market quotations.

## 6. Input Validation

The program accepts only menu options 1, 2, or 3. Vehicle ID and rental duration must be positive whole numbers. If a non-numeric value or a non-positive number is entered for these fields, the program clears the input error and asks again.

## 7. Testing

Test cases include a three-day car rental, a two-day bike rental, zero/invalid duration, invalid menu choice, and exit. Sample illustrative runs are included in sample-output.txt. Actual screenshots should be captured after compiling and running the program locally.

## 8. Conclusion

The project demonstrates how inheritance and runtime polymorphism can be used to model related vehicle types with different rental rules. The menu and validation make the application straightforward to use.

*Student name, roll number, course, and faculty details can be added before final submission if required by the instructor.*